



EDR · Compliance · Endpoint Posture

Internal Documentation

EDR Compliance Dashboard

Technical Reference

Laravel 13

PHP 8.3+

Next.js 16

React 19

MySQL 8

Sanctum SPA

Argon2id

AES-256-GCM

Version: 1.0

Last updated: June 25, 2026

Audience: Engineering, Security, Operations

Contents

1	1. Overview
2	2. Architecture
3	3. Tech stack
4	4. Project layout
5	5. Data model & database schema
6	6. Authentication & session
7	7. Authorization (role-based access control)
8	8. Multi-factor authentication (MFA)
9	9. API source connectors
10	10. Ingestion pipeline
11	11. Insights engine
12	12. Dashboard system
13	13. Widget catalog & customization
14	14. Widget drill-down
15	15. Dashboard assignment to viewers
16	16. Email notification system
17	17. Tech stack auditor & CVE scanner
18	18. Audit trail
19	19. System Health page
20	20. API reference
21	21. Frontend architecture
22	22. Environment configuration
23	23. Deployment
24	24. Update procedure
25	25. Operations & gotchas

[26 26. Extending the system](#)

[27 Appendix A — Default users \(seeded\)](#)

[28 Appendix B — Useful local commands](#)

1. Overview

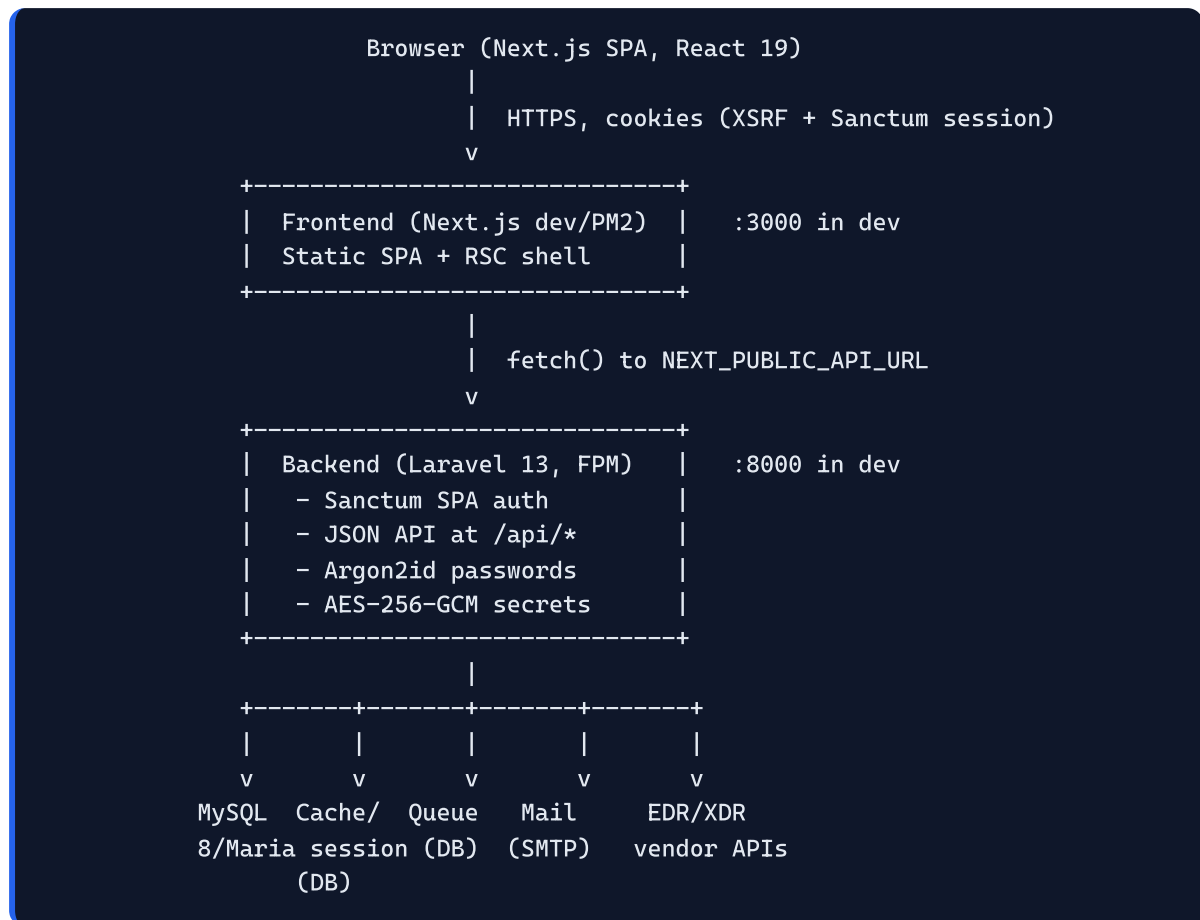
The platform pulls endpoint inventory from one or more EDR / XDR / SIEM APIs (CrowdStrike, Microsoft Defender, SentinelOne, Wazuh, Trend Micro, Cortex, Cisco AMP, Elastic, Sophos, or a generic connector), normalizes the records into a uniform endpoint shape, persists periodic snapshots, and renders the estate through customizable dashboards.

It is multi-tenant in spirit (each user owns their own sources, sites, and dashboards) with an admin-controlled overlay for sharing dashboards to read-only viewers, managing users, and configuring system-wide settings like SMTP and email templates.

Primary capabilities

- **Connector framework** — pluggable HTTP connectors with field mapping, encryption-at-rest for credentials, scheduled and on-demand refresh.
- **Snapshot history** — each refresh persists a new snapshot, so trends are computed from real history rather than live polling.
- **Custom dashboard builder** — drag-and-resize grid; pie / donut / bar / line / area / stat / gauge / table widgets with rich per-type customization (sort, top-N, palette, thresholds, granularity, etc.).
- **Drill-down** — click any chart slice → modal listing the underlying endpoints.
- **Dashboard sharing** — admins assign dashboards to viewers; updates the admin makes propagate live; viewers cannot modify.
- **Role-based access control** — admin / analyst / viewer with explicit permission matrix shown in-app.
- **MFA** — TOTP authenticators, admin-controlled enrollment for non-admins.
- **Email notifications** — 10 event types, fully editable templates, per-user opt-in, runtime SMTP config.
- **Tech stack audit** — live CVE scan of installed PHP + npm packages with CVE / GHSA identifiers, severity, CVSS scores.
- **Audit logging** — every administrative and security-relevant action.
- **Session security** — argon2id hashing, IP baselining, brute-force lockout, new-IP flagging, comprehensive sign-in audit.

2. Architecture



Request flow (typical browser action)

1. The frontend mounts `<AuthProvider>` which calls `GET /api/me`. Without a session cookie this returns 401 → the user is redirected to `/login`.
1. Login posts to `POST /api/login`. Successful login writes a session cookie and (when applicable) prompts for MFA.
1. Every subsequent API call sends the session cookie and the `X-XSRF-TOKEN` header (read from the `XSRF-TOKEN` cookie that Sanctum sets).
1. Laravel resolves the user via Sanctum's `EnsureFrontendRequestsAreStateful` middleware, then dispatches to the controller.
1. Controllers consult Eloquent models (and a small set of service classes for crypto, mail, MFA, ingestion, rule evaluation, notifications, and tech-stack auditing) and return JSON.
1. React Query on the frontend caches responses keyed by query parameters

and re-validates on interval / focus.

Why two separate repos

Splitting decouples the deploy lifecycle (you can hot-rebuild the SPA without touching PHP-FPM), avoids accidentally shipping `vendor/` and `node_modules/` into one tree, and lets you scale them independently — for example serving the SPA from a CDN while the API runs on a VPS.

3. Tech stack

LAYER	CHOICE	NOTES
Backend language	PHP 8.3+	composer.json requires ^8.3 ; dev box runs 8.5
Backend framework	Laravel 13.8	Standard scaffold + Sanctum 4
API auth	Laravel Sanctum (SPA cookie mode)	Not bearer tokens — uses session cookies + CSRF
Password hashing	Argon2id	HASH_DRIVER=argon2id , 65536 KB memory, 4 iterations
Secret encryption	AES-256-GCM	Custom SecretBox service, dedicated DATA_ENCRYPTION_KEY
MFA	TOTP (RFC 6238)	pragmarx/google2fa + recovery codes
Database	MySQL 8 / MariaDB	utf8mb4 unicode collation
Cache / session / queue	Database driver	Avoids extra infrastructure; swap to Redis trivially
Mail	Symfony Mailer via Laravel	SMTP transport configured at runtime from DB row
Vuln advisories	composer audit + npm audit	Live Packagist + GitHub Advisory DB
Frontend framework	Next.js 16.2 + React 19	App Router, (app) route group
State & data	TanStack Query v5	Server state caching
Charts	Recharts 3	Bar / Line / Area / Pie / RadialBar
Grid layout	react-grid-layout	Drag / resize for the builder
Styling	Tailwind CSS v4 + CVA	cn() helper for class composition
Icons	lucide-react	
QR codes	qrcode.react	MFA enrollment

4. Project layout

```

ComplaineDashboardFrontend/      (working folder - not a deployed unit)
├── Backend/                     # Laravel app - pushed to
ComplaineDashboardBackend repo
|   ├── app/
|   |   ├── Http/Controllers/Api/ # AuthController, AdminController,
DashboardController,           # ApiSourceController, InsightsController,
|   |   |                           # MfaController, NotificationController,
HealthController,              # DataController, PasswordController
|   |   |                           # EnsureRole, TrackActivity
|   |   ├── Http/Middleware/      # GenericNotificationMail
|   |   ├── Mail/                 # User, ApiSource, Endpoint, Snapshot,
|   |   ├── Models/               # Site, KnownIp, LoginEvent, AuditLog,
Dashboard,                      # NotificationTemplate,
|   |   |                           # NotificationLog, SourceRun
MailSettings,                  # Boots MailConfigurator
|   |   |                           # ConnectorFactory + per-vendor classes
NotificationSubscription,      # AES-256-GCM
|   |   |                           # IngestService, Normalizer, Summarizer,
|   |   ├── Providers/AppServiceProvider.php
|   |   ├── Services/
|   |   |   ├── Connectors/
|   |   |   ├── Crypto/SecretBox.php
|   |   |   └── Ingest/
RuleEvaluator
|   |   |   ├── Notifications/
|   |   |   └── Security/
MfaService,
|   |   |   ├── Support/DefaultDashboard.php
|   |   |   ├── database/
|   |   |   ├── migrations/
|   |   |   ├── audit_logs,
|   |   |   ├── endpoints,
|   |   |   ├── mail_settings,
|   |   |   ├── seeders/
|   |   |   ├── resources/views/emails/
|   |   |   ├── routes/api.php
|   |   |   ├── config/
|   |   |   └── public/index.php
└── Frontend/                   # Next.js app - pushed to
ComplaineDashboardFrontend repo
    ├── src/
    |   ├── app/
    |   |   ├── (app)/          # Route group - every page wrapped in
<Shell>
|   |   |   ├── dashboard/      # Main dashboard
|   |   |   └── builder/        # Customize dashboard (manager-only)

```

```

├── data/                # Endpoint table view (manager-only)
├── sources/             # Source CRUD (manager-only)
├── admin/               # Admin CMS
├── health/              # System Health
├── settings/            # Per-user settings
├── login/               # Login (+ MFA challenge)
├── providers.tsx        # QueryClient + Toast + Auth contexts
├── components/
├── ui/                  # Button, Card, Dialog, Select, Input,
Badge, Tabs, ...
├── notifications/      # MailSettingsTab, TemplatesTab,
                        # NotificationLogsTab, MyNotificationsCard
├── dashboard-view.tsx  # WidgetCard + DashboardView grid renderer
├── widget.tsx          # Per-type widget renderer + drill click
handlers
├── endpoint-table.tsx   # Reusable paginated endpoint table
├── endpoint-drill-dialog.tsx
├── roles-permissions-card.tsx
├── role-guard.tsx       # In-page route guard
├── tech-stack-panel.tsx
├── source-bar.tsx / source-wizard.tsx
├── scope-bar.tsx        # Scope selector (all / site / source)
├── shell.tsx            # Sidebar nav + topbar
├── page-header.tsx
├── lib/
├── api.ts               # fetch wrapper + CSRF handling
├── auth.ts              # AuthContext + useAuth()
├── queries.ts           # All React Query hooks
├── types.ts             # All TypeScript types
├── format.ts            # titleCase, palettes, badge variants
├── toast.tsx
├── use-scope.ts
├── use-active-source.ts
├── next.config.ts
├── package.json

```

5. Data model & database schema

Tables

USERS

The single user table. Roles encoded as a string column. Sensitive fields (`password` , MFA secrets, recovery codes) are hidden from JSON serialization.

COLUMN	TYPE	NOTES
<code>id</code>	bigint PK	
<code>name</code> , <code>email</code>	string	email unique
<code>password</code>	string	argon2id hash
<code>role</code>	string	<code>admin</code> / <code>analyst</code> / <code>viewer</code>
<code>is_active</code>	bool	when false, login is rejected
<code>mfa_enabled</code> , <code>mfa_secret</code> , <code>mfa_recovery_codes</code> , <code>mfa_confirmed_at</code>	mixed	TOTP enrollment state; recovery codes hashed at rest
<code>mfa_required</code>	bool	admin sets true to allow / require user enrollment
<code>failed_login_attempts</code> , <code>locked_until</code>	int / datetime	brute-force throttle
<code>last_login_at</code> , <code>last_login_ip</code> , <code>current_ip</code> , <code>last_seen_at</code>	mixed	session tracking
<code>ip_flagged</code> , <code>must_change_password</code>	bool	security flags surfaced in UI
<code>preferences</code>	json	per-user UI preferences

PERSONAL_ACCESS_TOKENS

Stock Sanctum table — present but unused (we use SPA cookie mode).

LOGIN_EVENTS

Append-only audit of every sign-in attempt (successful or not).

COLUMN	NOTES
<code>user_id</code>	nullable — anonymous attempts also recorded
<code>email</code>	the email that was tried
<code>ip_address</code> , <code>user_agent</code>	
<code>successful</code> , <code>is_new_ip</code>	booleans
<code>failure_reason</code>	string
<code>created_at</code>	

KNOWN_IPS

IP baseline per user. First IP after enrollment is auto-trusted; later IPs trigger `users.ip_flagged = true` and the `login.new_ip` notification.

COLUMN	
<code>user_id</code> , <code>ip_address</code>	unique together
<code>trusted</code>	bool
<code>login_count</code> , <code>first_seen_at</code> , <code>last_seen_at</code>	

AUDIT_LOGS

General-purpose admin / security audit trail. Written via `AuditLogger` .

COLUMN	NOTES
<code>user_id</code>	actor (nullable for system events)
<code>action</code>	e.g. <code>dashboard.assigned</code> , <code>admin.user_deleted</code> , <code>mfa.disabled</code>
<code>target_type</code> , <code>target_id</code>	resource the action acted upon
<code>meta</code>	JSON payload (role, old/new value, etc.)
<code>ip_address</code> , <code>created_at</code>	

SITES

A grouping of `api_sources` (e.g. "London office") to scope dashboards without selecting a specific source.

API_SOURCES

Connector definitions. Credentials are encrypted with `SecretBox` .

COLUMN	NOTES
<code>user_id</code>	owner
<code>site_id</code>	nullable grouping
<code>name</code> , <code>vendor</code>	display + connector key
<code>base_url</code> , <code>auth_type</code>	HTTP target + bearer / <code>api_key_header</code> / basic / <code>oauth2_client_credentials</code>
<code>auth_config</code> , <code>request_config</code> , <code>field_mappings</code>	JSON — connector-specific
<code>secret_encrypted</code>	AES-256-GCM ciphertext when <code>secret_mode = saved</code>
<code>secret_mode</code>	<code>saved</code> (persist) or <code>per_login</code> (user re-enters each session)
<code>refresh_interval_minutes</code>	15..10080 (15 min minimum)
<code>is_enabled</code> , <code>last_status</code> , <code>last_error</code> , <code>last_run_at</code> , <code>latest_snapshot_id</code>	runtime status

SOURCE_RUNS

Per-ingestion attempt log: timing, record count, error message.

SNAPSHOTS

A point-in-time crystallization of a source's endpoint inventory. The serialized `summary` JSON is the pre-aggregated rollup the dashboard reads.

COLUMN	
<code>api_source_id</code> , <code>source_run_id</code>	
<code>captured_at</code>	when the API was queried
<code>endpoint_count</code>	
<code>summary</code>	JSON: total, online, stale, offline, compliant, non_compliant, compliance_pct, by_os{}, by_health{}, by_compliance{}, by_agent_version{}

ENDPOINTS

Normalized per-endpoint rows attached to a snapshot.

COLUMN	
<code>snapshot_id</code> , <code>api_source_id</code> , <code>external_id</code>	
<code>hostname</code> , <code>os_platform</code> , <code>os_version</code> , <code>agent_version</code>	
<code>health_status</code> , <code>compliance_status</code> , <code>is_isolated</code>	
<code>ip_address</code> , <code>mac_address</code> , <code>last_seen_at</code>	
<code>extra</code> , <code>raw</code>	JSON — connector-specific extras + raw record
<code>captured_at</code>	denormalized from snapshot for query speed

DASHBOARDS

Per-user dashboard layouts.

COLUMN	
<code>user_id</code>	owner
<code>api_source_id</code>	optional default scope binding
<code>name</code> , <code>is_default</code>	
<code>layout</code>	JSON array of <code>Widget</code> objects (see section 13)

DASHBOARD_USER (PIVOT)

Many-to-many: which dashboards have been assigned to which viewers.

COLUMN	
<code>dashboard_id</code> , <code>user_id</code>	unique together
<code>assigned_by_user_id</code>	the admin who assigned it
<code>created_at</code> , <code>updated_at</code>	

MAIL_SETTINGS

Single-row table holding runtime SMTP config.

COLUMN	NOTES
<code>transport</code>	<code>smtp</code> or <code>log</code>
<code>host</code> , <code>port</code> , <code>encryption</code>	
<code>username</code> , <code>password_encrypted</code>	password encrypted with <code>SecretBox</code>
<code>from_address</code> , <code>from_name</code> , <code>reply_to</code>	
<code>enabled</code>	master switch
<code>last_test_at</code> , <code>last_test_status</code> , <code>last_test_error</code>	for the UI status badge

NOTIFICATION_TEMPLATES

One row per event_key. Subject + HTML + optional plain-text body, plus an enabled flag. Seeded from `NotificationCatalog::events()` .

NOTIFICATION_SUBSCRIPTIONS

Per-user opt-in. Missing row = inherit role default.

NOTIFICATION_LOGS

Append-only record of every send attempt (queued / sent / failed / skipped), including the rendered subject, recipient, payload snapshot, and error.

Cascades

- `users` is the root of most cascades. Deleting a user cascades to their `dashboards` , `sites` , `api_sources` (and through them `source_runs` , `snapshots` , `endpoints`), `login_events` , `known_ips` , `notification_subscriptions` , and their pivot rows in `dashboard_user` .

- `audit_logs.user_id` is set to NULL on user delete (preserves the trail).

Migration history (chronological)

```
0001_01_01_000000 create_users_table
0001_01_01_000001 create_cache_table
0001_01_01_000002 create_jobs_table
2026_06_24_184640 create_personal_access_tokens_table
2026_06_24_190001 add_security_fields_to_users_table
2026_06_24_190002 create_login_events_table
2026_06_24_190003 create_known_ips_table
2026_06_24_190004 create_audit_logs_table
2026_06_24_190005 create_api_sources_table
2026_06_24_190006 create_source_runs_table
2026_06_24_190007 create_snapshots_table
2026_06_24_190008 create_endpoints_table
2026_06_24_190009 create_dashboards_table
2026_06_25_000001 create_sites_table
2026_06_25_000002 create_dashboard_user_table <-- dashboard sharing
2026_06_25_000003 create_mail_settings_table <-- runtime SMTP
2026_06_25_000004 create_notification_templates_table
2026_06_25_000005 create_notification_subscriptions_table
2026_06_25_000006 create_notification_logs_table
2026_06_25_000007 add_mfa_required_to_users
```


6. Authentication & session

Mode

Sanctum SPA cookie authentication. There are **no bearer tokens** in normal use. Authenticated state is held in a server-side session referenced by an `HttpOnly` cookie. CSRF is enforced by sending the `XSRF-TOKEN` cookie value back as the `X-XSRF-TOKEN` header on every mutating request.

Login flow

1. GET `/sanctum/csrf-cookie` (handled by Sanctum) sets `XSRF-TOKEN`.
2. POST `/api/login` with `email`, `password`. The server:
 - rejects if the user is `is_active=false` or `locked_until > now()`,
 - verifies password against the argon2id hash,
 - if MFA is enabled, returns `mfa_required: true` and a setup-token,
 - on success, calls `LoginSecurityService::registerSuccess()` which

tracks the IP, updates last-login fields, resets the failure counter, records a `LoginEvent`, writes an `audit_logs` row, and (if it was a new IP) dispatches the `login.new_ip` notification.

1. Failed logins call `registerFailure()` which increments `failed_login_attempts`, locks the account when the threshold is reached (and fires `account.locked`), and records the failed event. When attempts exceed the soft threshold (`max - 2`), the `login.failed_threshold` notification fires.

1. The frontend's `<AuthProvider>` invalidates its `["me"]` query and redirects either to `/login` (MFA still owed) or `/dashboard`.

Frontend fetch wrapper

`src/lib/api.ts` exposes `api.get/post/put/del()`. It:

- lazily calls `/sanctum/csrf-cookie` before the first mutating request,
- reads `XSRF-TOKEN` cookie and sets `X-XSRF-TOKEN` header automatically,
- sends `credentials: include` so the session cookie flows,
- throws a typed `ApiError` with `.status`, `.errors`, `.firstError` so UI

toasts can surface validation messages cleanly.

Session, cache, queue drivers

All default to `database` . `SESSION_ENCRYPT=true` so the session blob on disk is opaque. No Redis required out of the box.

7. Authorization (role-based access control)

Three roles, defined as constants on `User` :

CONSTANT	VALUE	CAPABILITY
<code>ROLE_ADMIN</code>	<code>admin</code>	Full control
<code>ROLE_ANALYST</code>	<code>analyst</code>	Sources + own dashboards, no user / system admin
<code>ROLE_VIEWER</code>	<code>viewer</code>	Read-only on assigned dashboards

Helpers on `User`

```
$user->isAdmin();           // role === 'admin'  
$user->canManage();         // role in [admin, analyst]  
$user->hasRole(...$roles);  // generic check
```

Server-side enforcement

- Routes under `Route::middleware('role:admin')->prefix('admin')` use the `EnsureRole` middleware to return 403 if the user is not an admin.
- Other write actions (dashboard CRUD, source CRUD, site CRUD) check `abort_unless($user->canManage(), 403)` inside the controller.
- The MFA controller gates self-service endpoints behind `user.is_admin || user.mfa_required`.
- Per-resource ownership is enforced inline (e.g. dashboards only

modifiable by their `user_id` ; sources only readable by their owner; insights resolved against the user's own `apiSources()` unless the request includes a `dashboard_id` that the user has read access to).

Client-side enforcement

- `<Shell>` nav filters items by `user.is_admin` / `user.can_manage`.
- `<RoleGuard require="manage|admin">` wraps manage-only pages (`/builder` , `/sources` , `/sources/new` , `/sources/[id]/edit` , `/data`). It shows a permission notice and redirects to `/` after a short delay.
- Inside dialogs, buttons that require capability are conditionally

rendered (e.g. "Delete user" only renders if not viewing self).

Permission matrix (also rendered in-app)

PERMISSION	ADMIN	ANALYST	VIEWER
View assigned dashboards	✓	✓	✓
Build / edit own dashboards	✓	✓	—
Assign dashboards to others	✓	—	—
Browse endpoint data tables	✓	✓	—
Add / edit / delete connectors	✓	✓	—
Trigger on-demand refresh	✓	✓	—
Create users, set roles, reset passwords	✓	—	—
Delete users	✓	—	—
Require / reset another user's MFA	✓	—	—
View login activity & audit log	✓	—	—
Change own password	✓	✓	✓
Self-enroll MFA	✓	only if <code>mfa_required</code>	only if <code>mfa_required</code>
Disable own MFA	✓	—	—
Receive email notifications	✓	✓	✓
Configure SMTP / templates	✓	—	—
View tech stack & CVE scan	✓	—	—

8. Multi-factor authentication (MFA)

Mechanism

TOTP (RFC 6238). The provisioning URI is rendered as a QR code client-side (`qrcode.react`). Recovery codes are generated once at enrollment and stored hashed.

Self-enrollment gating

Self-enrollment is restricted. `MfaController` checks at the top of `setup()`, `confirm()`, and `regenerateRecoveryCodes()`:

```
abort_unless(
  $user->isAdmin() || $user->mfa_required,
  403,
  'MFA enrollment is controlled by your administrator.'
);
```

`disable()` is admin-only. Non-admins who want to remove MFA must ask an admin to call `POST /api/admin/users/{user}/disable-mfa`.

Flow for a non-admin user

1. Admin opens **Admin** → **Users** → **Manage** on the user.
2. Admin clicks **Require MFA** → `PUT /api/admin/users/{user}/mfa-required`

with `{required: true}` and a `dashboard.assigned` -style audit log entry.

1. User signs in. Their `mfa_required = true` flag means the Settings page's MFA card now shows the enable button.
1. User clicks Enable, scans the QR, enters a 6-digit code → MFA active.
2. From now on, every login presents the MFA challenge after password.

Admin-side controls

- **Require MFA** — toggle `mfa_required`.
 - **Reset MFA** — wipes the authenticator (user must re-enroll).
 - **Cancel MFA requirement** — clears `mfa_required` if they haven't enrolled yet.
-

9. API source connectors

A connector is a thin class that:

- Receives an `ApiSource` and a decrypted secret.
- Builds a request from `auth_type` + `auth_config` + `request_config`.
- Returns an iterable of raw records.

The connectors live under `app/Services/Connectors/`. Each implements the same interface so the rest of the system doesn't care which vendor it is.

`ConnectorFactory::make($source)` resolves the right class by `vendor`.

Vendors supported

`generic`, `crowdstrike`, `defender`, `sentinelone`, `wazuh`, `trendmicro`, `cortex`, `cisco_amp`, `elastic`, `sophos`.

Authentication modes

- `bearer` — `Authorization: Bearer {secret}`
- `api_key_header` — custom header name from `auth_config.header`
- `basic` — `base64(username:password)` where `password = secret`
- `oauth2_client_credentials` — exchange `client_id` + secret for a token

Field mapping

The Normalizer applies `field_mappings` to convert vendor-specific JSON keys into the canonical endpoint shape. Example for CrowdStrike:

```
{
  "hostname": "hostname",
  "os_platform": "platform_name",
  "os_version": "os_version",
  "agent_version": "agent_version",
  "health_status": "_derived:online_status",
  "ip_address": "local_ip"
}
```

A `_derived:*` prefix triggers special logic in the Normalizer (e.g. mapping vendor status strings to `online` / `stale` / `offline`).

Secret modes

- `saved` — secret is encrypted with `SecretBox` (AES-256-GCM, keyed by `DATA_ENCRYPTION_KEY`) and stored. Scheduler can run without user interaction.

- `per_login` — secret is held in session memory only. Refreshes only

succeed while at least one operator session has unlocked it. The `KnownIps` and `LoginEvents` reflect this state.

Wizard UX

`source-wizard.tsx` is a 4-step wizard:

1. **Vendor** — pick a preset that pre-fills `base_url`, `auth_type`, `auth_config`, `request_config`, and `field_mappings`. A "Generic" option lets you type everything by hand.
1. **Connect** — credentials + test (`POST /api/sources/test`).
2. **Map fields** — adjust the mapping table.
3. **Schedule** — pick **Preset** (from `config('security.refresh_intervals')`) or **Custom** (any integer between 15 and 10080 minutes), key handling mode, optional site grouping.

On-demand refresh

`POST /api/sources/{source}/refresh` queues an immediate ingest run. The button appears next to each source in the list.

10. Ingestion pipeline

`IngestService::run($source, $secret, $trigger)` is the entry point. It:

1. Creates a `SourceRun` row with status `running`.
2. Calls `Connector::fetch()` to get raw records.
3. Pipes each record through `Normalizer::normalize($record, $mappings)`.
4. Calls `Summarizer::summarize($normalized)` to build the per-snapshot

rollup (total, by_os, by_health, by_compliance, by_agent_version, compliance_pct).

1. Inside a DB transaction, writes a `Snapshot`, bulk-inserts the

`endpoints`, and updates `api_sources` with `latest_snapshot_id`, `last_run_at`, `last_status='success'`, `last_error=null`.

1. Prunes old endpoint detail beyond the retention window to keep table sizes bounded (still retains the rolled-up snapshot for trends).

1. On exception: marks `last_status='failed'`, stores the error, dispatches `source.refresh_failed` notification (only on the success→failed transition, not on every retry).

1. On a recovery (failed→success), dispatches `source.refresh_recovered`.

Notification dispatch happens in the service, so all triggers — CLI, scheduler, on-demand refresh, queued job — fire identically.

11. Insights engine

`InsightsController` is the only place that reads endpoint / snapshot data for the dashboards. It has five endpoints, all scope-aware and dashboard-aware.

Scope

A request can pass `scope = all | site:{id} | source:{id}`. The `resolveSources()` helper:

- If `dashboard_id` is present and the requesting user can read it, uses the dashboard owner's sources (so viewers see real data on assigned dashboards).
- Otherwise uses the requesting user's own sources.
- Applies the scope filter on top (site or single-source narrowing).

`GET /api/insights/summary`

Sums every source's most recent snapshot summary in scope:

```
{
  "summary": {
    "total": 152, "online": 124, "stale": 18, "offline": 10,
    "compliant": 138, "non_compliant": 14, "compliance_pct": 90.8,
    "by_os": {"Windows": 90, "Linux": 35, "macOS": 27},
    "by_health": { ... }, "by_compliance": { ... }, "by_agent_version": { ... },
    "agent_versions": 12
  },
  "captured_at": "...",
  "endpoint_count": 152,
  "source_count": 3,
  "last_error": null // or "1 source failed: Corp CrowdStrike"
}
```

`GET /api/insights/aggregate?field=...`

Groups the latest snapshot's endpoints by an allowed field (`hostname` , `os_platform` , `os_version` , `agent_version` , `health_status` , `compliance_status` , `ip_address` , `mac_address` , `last_seen_at` , `external_id` , `is_isolated`) and returns `{ field, buckets: [{label, value}] }` sorted by count desc, limited to 30.

`GET /api/insights/trends`

Returns daily rollups for the last N (default 90, max 365) days. For each day-source pair it keeps the latest snapshot to avoid double-counting multiple pulls. Then sums across sources.

```
{ "series": [
  {"captured_at": "2026-06-20", "total": 150, "online": 122, "stale": 18,
    "offline": 10, "compliant": 136, "non_compliant": 14, "compliance_pct":
    90.7},
  ...
]}
```

GET /api/insights/data

Paginated endpoints with filters. Filterable fields: `os_platform`, `os_version`, `health_status`, `compliance_status`, `agent_version`, `ip_address`, `mac_address`, `is_isolated`. Plus `search` (LIKE across hostname / IP / `os_version` / `external_id`) and `sort` / `dir`.

POST /api/insights/evaluate

Evaluates a custom rule against the in-scope endpoints and returns `{count, total, pct}`. Powers the rule-based stat / gauge widgets.

POST /api/insights/rule-data

Same rule, but returns the paginated endpoints (used by drill-down on rule-based widgets).

Rule engine (`RuleEvaluator`)

A rule is `{match: 'all'|'any', conditions: [{field, op, value}, ...]}`.

Operators per field-type:

- **duration** (`last_seen_days`) — `gt`, `lt` (with auto NULL handling)
- **enum** (health, compliance, OS, etc.) — `eq`, `neq`, `in`
- **bool** (`is_isolated`) — `eq`
- **text** (hostname, IP, `agent_version`, ...) — `eq`, `neq`, `contains`,

`not_contains`, `is_empty`, `not_empty`

Internally, `buildQuery()` produces an Eloquent builder so both `count()` (for evaluate) and the rule-data endpoint share logic.

12. Dashboard system

Storage

A dashboard is a row in `dashboards` with a JSON `layout` array. Each widget in the array is self-contained — there is no separate widgets table.

Widget shape

```
interface Widget {
  id: string;           // generated client-side
  type: "stat" | "gauge" | "pie" | "donut" | "bar" | "line" | "area" | "table";
  title: string;
  config: WidgetConfig; // see section 13
  x: number; y: number; w: number; h: number; // 12-col grid units
}
```

Default dashboard

`DefaultDashboard::layout()` defines what new users see on first login: 4 stat widgets, a pie (OS), a donut (health), two bars (compliance, agent version), a line (compliance trend), and an endpoint table.

Builder page (`/builder`)

- `react-grid-layout/legacy` provides drag + resize.
- Top toolbar: **Add widget**, **Revert**, **Save**.
- **Add widget** opens a dialog with a preset dropdown (10 starter recipes) and per-type configuration controls.
- **Save** sends `PUT /api/dashboards/{id}` with the full layout.

The builder is wrapped in `<RoleGuard require="manage">`.

Rendering pipeline

`DashboardPage` → `<DashboardView layout={...} dashboardId={...}>` → for each widget renders a `<WidgetCard>` which:

1. Renders title + (in builder) actions
2. Mounts `<WidgetBody widget scope summary trends dashboardId onDrill>`
3. Owns the drill-dialog state and mounts `<EndpointDrillDialog>` when set

13. Widget catalog & customization

Per-type controls

TYPE	KNOBS
stat	metric (preset: total / online / stale / offline / compliant / non_compliant / compliance_pct) OR custom rule; display=count/percent for rule mode; thresholds (good $\geq$ / warn $\geq$ / direction); unit suffix
gauge	metric OR rule; thresholds (color the value); unit suffix
pie, donut	group-by field; sort (value desc/asc, label, server); top-N + "Other" bucket; palette; show legend; show values
bar	same as pie + horizontal toggle
line, area	series multi-select; time range (7/14/30/90/all); granularity (day/week, client-side aggregation); smooth toggle; Y-axis suffix; palette; show legend
table	(no config)

Sort + Top-N + "Other"

Done client-side in `shapeBuckets()` in `widget.tsx`:

```
buckets.sort(by sort mode);
if (topN > 0 && buckets.length > topN) {
  const head = buckets.slice(0, topN);
  if (showOther) head.push({label: "Other", value: sumOfRest});
  return head;
}
```

Thresholds (stat / gauge)

```
direction = higher_is_better:
  value >= good  → text/value colored success (green)
  value >= warn  → warning (amber)
  else          → destructive (red)
direction = lower_is_better:
  ≤ good → success; ≤ warn → warning; else → destructive
```

Time range + granularity (line / area)

Client-side filter on `captured_at >= now - rangeDays`, then optional Monday-anchored weekly grouping that averages numeric series across each week. This lets you switch a 90-

day line from noisy daily ticks to clean weekly trend without touching the backend.

Palette presets

Defined in `lib/format.ts` :

KEY	COLORS
<code>default</code>	chart-1..5 (theme variables)
<code>ocean</code>	sky / cyan / teal blues
<code>sunset</code>	orange / rose / purple / yellow / red
<code>forest</code>	greens
<code>mono</code>	slate scale

Preset recipes

The Add-widget dialog has a "Load preset" dropdown with 10 ready-to-use configurations (e.g. "Donut • Health status", "Bar • Agent versions (top 10, horizontal)", "Area • Health counts (90d, weekly)"). Picking a preset fills every field in the form so the user can fine-tune.

14. Widget drill-down

Click any pie/donut slice, bar, stat, or gauge → opens `<EndpointDrillDialog>` with the matching endpoints.

Click → criteria mapping

SOURCE	RESULTING CRITERIA
Pie/donut/bar slice	<code>{filter: {field: <widget.config.field>, value: <slice.label>}}</code>
"Other" rollup	Not clickable (it's an aggregate, no single filter)
Stat with metric=online/stale/offline	<code>{filter: {field: "health_status", value: <metric>}}</code>
Stat with metric=compliant/non_compliant	<code>{filter: {field: "compliance_status", value: <metric>}}</code>
Stat with metric=total	<code>{}</code> (no filter — all endpoints in scope)
Stat with metric=compliance_pct	Not clickable
Stat/Gauge in rule mode	<code>{rule: <widget.config.rule>}</code>

Backend support

- `GET /api/insights/data` accepts the extended set of field filters and honors `dashboard_id` so viewer drill-down resolves against the dashboard owner's sources.
- `POST /api/insights/rule-data` mirrors `evaluate` but returns paginated endpoints instead of counts.

Dialog features

- Filtered table (hostname / OS / OS version / agent / health / compliance / last-seen / IP).
- Search (disabled in rule mode — rules are already specific).
- Sort by hostname / os / agent / last_seen_at.
- 25 per page, prev/next pagination.

- **Export CSV** of the current filtered slice.

Builder mode

The builder passes `drillEnabled={{false}}` to `<WidgetCard>` so clicking inside the layout editor doesn't pop modals.

15. Dashboard assignment to viewers

Model

`dashboards` belong to one user. `dashboard_user` pivot stores many-to-many shares (`dashboard_id` , `user_id` , `assigned_by_user_id`).

Behavior

- **Admin** assigns one of their own dashboards to a viewer via the Admin → User → Manage dialog (Assigned Dashboards section).
- The viewer immediately sees the dashboard on `/dashboard` (the `useDefaultDashboard()` query returns it).
- When the admin edits the layout (`PUT /api/dashboards/{id}`), the viewer sees the update on next page load — assignments are **live links**, not snapshots.
- The viewer cannot modify (the dashboard ownership check in `authorizeWrite()` returns 403).
- Every viewer dashboard load writes a `dashboard.viewed` row to `audit_logs` so admins can see who is using which dashboard.

Multi-assignment

If a viewer has more than one assigned dashboard, a switcher dropdown appears at the top of `/dashboard` . The dashboard page maintains `activeId` state and fetches the selected one via `useDashboard(id)` .

Viewer-only banner

When a read-only dashboard is selected, a small primary-tinted strip displays: `*"View-only dashboard assigned by {owner}. Updates from the owner appear automatically."*`

How insights resolve for viewers

`InsightsController::resolveSources()` checks for a `dashboard_id` query param. If present and the user can read the dashboard (owner OR assigned), sources are pulled from the **dashboard owner's** `apiSources()` . This is how a viewer with no sources of their own sees actual data on an admin's dashboard.

16. Email notification system

Design summary

- **10 event types** across 5 categories.
- **Templates live in DB** (`notification_templates`) — admins edit them

in the UI with live preview + test send.

- **Mail transport** configured at runtime from `mail_settings` row

(encrypted SMTP password). `MailConfigurator` overrides Laravel mail config in `AppServiceProvider::boot()` .

- **Per-user opt-in** — `notification_subscriptions` table. Missing row = use role default audience defined in the catalog.
- **Real-time triggers** — wired into existing flows (login, admin

actions, source ingest, tech-stack scan).

Event catalog

EVENT KEY	CATEGORY	DEFAULT AUDIENCE	TRIGGER
<code>login.new_ip</code>	security	admins	LoginSecurityService — fresh IP after a baseline exists
<code>login.failed_threshold</code>	security	admins	LoginSecurityService — N-2 failures (configurable)
<code>account.locked</code>	security	admins	LoginSecurityService — failed login that exceeds max
<code>account.mfa_disabled</code>	account	admins	AdminController::disableMfa
<code>account.password_reset</code>	account	the affected user	AdminController::resetPassword
<code>account.role_changed</code>	account	admins	AdminController::update
<code>dashboard.assigned</code>	dashboard	the assignee	AdminController::assignDashboard
<code>source.refresh_failed</code>	source	admins + analysts	IngestService — success→failed
<code>source.refresh_recovered</code>	source	admins + analysts	IngestService — failed→success
<code>vuln.new_advisory</code>	vulnerability	admins	TechStackAuditor — new advisory not seen before

Template engine

A safe "mustache-lite" renderer (`TemplateRenderer`). Templates use `{{ key }}` and `{{ nested.key }}` syntax — no control flow, no PHP, admins cannot execute code via templates.

Every template gets these shared variables in addition to its declared ones:

```
{{ app.name }}, {{ app.url }}
{{ event_key }}
{{ recipient.name }}, {{ recipient.email }}
```

Mail transport

`mail_settings` is a singleton row. `MailConfigurator::apply()` is called at boot from `AppServiceProvider` . If the row says `enabled=false` or `transport=log` (or the table

doesn't exist yet), `.env` defaults win and emails go to the log driver.

When `enabled=true` :

```
Config::set('mail.default', 'smtp');
Config::set('mail.mailers.smtp', [
    'transport' => 'smtp',
    'host' => $s->host, 'port' => $s->port,
    'encryption' => $s->encryption, 'username' => $s->username,
    'password' => $s->getPassword(), // decrypted via SecretBox
    'timeout' => 15,
]);
Config::set('mail.from', [...]);
```

Then `Mail::purge('smtp')` clears any cached mailer so the next send uses the fresh config.

Dispatch flow

1. A trigger calls `NotificationService::dispatch($eventKey, $payload, $targetUser=null)`.

1. The service loads the template — returns silently if disabled / missing.

2. Recipient resolution:

- If `$targetUser` was supplied, that user is the only recipient (still respects their personal subscription preference).
- Otherwise: iterate active users, include each whose effective subscription is enabled (explicit row OR role default).

1. For each recipient: render subject + HTML + text with merged payload (event vars + `app.*`, `event_key`, `recipient.*`).

1. Write a `notification_logs` row with status `queued`.

2. `Mail::to($email)->send(new GenericNotificationMail(...))`.

3. On success: status= `sent`, `sent_at=now`. On failure: status= `failed`,

error message truncated to 1000 chars and written to `laravel.log`.

Sending is **synchronous** in the request that triggered it. For high- volume scenarios, the `GenericNotificationMail` class can be made to implement `ShouldQueue` and dispatch will queue rather than block.

Admin UI

- **Mail Settings tab** — transport (SMTP / log), full SMTP form, sender identity, master enable, Send test button with status badge ("Verified", "Last test failed").

- **Email Templates tab** — list grouped by category with severity badges.

Click Edit to open a dual-pane editor: subject + HTML body + plain-text fallback on the left; available variables (clickable to insert), default audience, and live preview on the right.

Buttons: Save, Preview, Reset to default, Send test.

- **Notification Log tab** — last 200 sends with status, event, recipient,

subject, timestamp. Failed rows show the error inline.

User UI

- **Settings → Email notifications** — every event listed with a toggle.

Defaults per role are indicated (e.g. "default: on"). Saving creates explicit subscription rows.

17. Tech stack auditor & CVE scanner

What it does

On demand (admin clicks Refresh) or on page visit (with 1h cache), the `TechStackAuditor` produces a snapshot containing:

- Runtime versions (PHP, Laravel, Node, npm, Composer, DB).
- Every installed PHP package from `composer.lock`.
- Every installed npm package from `Frontend/package-lock.json`.
- Security advisories attached to packages, sourced live from:
- `composer audit --format=json --no-interaction --locked` →

Packagist advisories DB.

- `npm audit --json --omit=dev` (run from the Frontend folder) →

GitHub Advisory DB.

Each advisory is normalized to `{id, cve, title, severity, affected_versions, cvss, url}`. For npm advisories where no CVE is listed, the GHSA ID is extracted from the advisory URL.

Output shape

```
{
  "generated_at": "...",
  "runtime": [
    {"name": "PHP", "version": "8.5.7", "ecosystem": "runtime"},
    ...
  ],
  "totals": {"packages": 598, "php": 113, "npm": 485, "vulnerable": 1},
  "advisories": 1,
  "packages": [
    {
      "ecosystem": "npm", "name": "postcss", "version": "8.5.15",
      "dev": false, "advisories": [
        {"id": "GHSA-QX2V-QP2M-JG93", "cve": "GHSA-QX2V-QP2M-JG93",
          "title": "PostCSS has XSS via Unescaped </style>...",
          "severity": "moderate", "affected_versions": "<8.5.10",
          "cvss": 6.1, "url": "https://github.com/advisories/..."}
      ],
      "highest_severity": "moderate"
    },
    ...sorted vulnerable first, then alphabetical
  ],
  "errors": [...]
}
```

Caching

`Cache::remember('health:tech-stack', 3600, ...)` . Force refresh by calling `snapshot(true)` or `GET /api/health/stack?refresh=1` .

Notification side effect

After every successful snapshot, `notifyNewAdvisories()` compares the current set of `(ecosystem|name|version|cve)` keys against a persistent "seen" set in cache. New entries trigger `vuln.new_advisory` notifications.

The very first scan only baselines the seen set — it does not spam emails for every existing advisory.

Admin UI panel

Located on the System Health page, admin-only. Features:

- Runtime strip across the top.
- Status badge: green "No known CVEs" or red "N vulnerable / M CVEs".
- Search by name / version.
- Filter by ecosystem (PHP / npm), severity (vulnerable-only, critical,

high, moderate, low), and include-dev toggle.

- Table: package, version, ecosystem, status, advisories. Vulnerable rows

red-tinted with CVE badge, CVSS score, affected-version range, and "details" link to the advisory.

- Refresh button that calls the endpoint with `refresh=1` .

Operational requirements

- `composer` and `npm` must be on the PATH that PHP-FPM uses.
- Outbound network from the server (to `packagist.org` and `registry.npmjs.org`).
- 60s timeout per audit. If either fails, the error appears in a yellow

banner on the panel and the other side's results still display.

- The auditor expects `Frontend/` to be a sibling of `Backend/` . If your deploy splits them differently, override `frontendPath()` .
-

18. Audit trail

`AuditLogger::log($action, ?$user, ?$target, $meta=[], ?$ip=null)` writes to `audit_logs`. The frontend Audit Log tab queries the last 200 entries.

Actions emitted

SOURCE	ACTION
Auth	<code>auth.login</code> , <code>auth.login_failed</code> , <code>auth.logout</code>
MFA	<code>mfa.enabled</code> , <code>mfa.disabled</code> , <code>mfa.recovery_codes_regenerated</code> , <code>admin.mfa_disabled</code> , <code>admin.mfa_required_set</code>
Account admin	<code>admin.user_created</code> , <code>admin.user_updated</code> , <code>admin.user_deleted</code> , <code>admin.password_reset</code> , <code>admin.user_unlocked</code> , <code>admin.ip_flag_cleared</code>
Sources	(covered via <code>SourceRun</code> rows + <code>last_status</code> on the source)
Dashboards	<code>dashboard.created</code> , <code>dashboard.updated</code> , <code>dashboard.deleted</code> , <code>dashboard.assigned</code> , <code>dashboard.unassigned</code> , <code>dashboard.viewed</code>
Notifications	<code>notification.template_updated</code> , <code>notification.subscriptions_updated_for_user</code> , <code>mail.settings_updated</code>

19. System Health page

Stock panels (visible to all authenticated users):

- **Application** — name, env, Laravel version, PHP version.
- **Database** — driver + ping latency.
- **Cache** — driver + write/read round-trip OK.
- **Queue** — connection + pending / failed counts (from `jobs` / `failed_jobs` tables).

- **Scheduler** — last cache stamp from a scheduled task; warns if older than 10 min.

- **Sources** — totals by state (enabled, failing, stale, per-login).
- **Security Posture** — user count, MFA adoption %, flagged IPs, locked accounts, online-now count.

Admin-only panel:

- **Tech Stack & Vulnerabilities** (see section 17).

Auto-refresh every 30s.

20. API reference

All routes are under `/api`. Authenticated routes use Sanctum SPA cookies.

Public

METHOD	PATH	NOTES
POST	<code>/login</code>	email + password → session + (maybe) MFA challenge
POST	<code>/login/mfa</code>	submit TOTP code

Authenticated (any role)

METHOD	PATH
GET	<code>/me</code>
POST	<code>/logout</code>
PUT	<code>/password</code>
POST	<code>/mfa/setup</code> (admins / mfa-required only)
POST	<code>/mfa/confirm</code> (admins / mfa-required only)
POST	<code>/mfa/disable</code> (admins only)
POST	<code>/mfa/recovery-codes</code> (admins / mfa-required only)
GET	<code>/notification-subscriptions</code>
PUT	<code>/notification-subscriptions</code>
GET	<code>/dashboards</code>
GET	<code>/dashboards/default</code>
GET	<code>/dashboards/{dashboard}</code> (owner or assignee)
GET	<code>/insights/summary</code>
GET	<code>/insights/aggregate</code>
GET	<code>/insights/trends</code>
GET	<code>/insights/data</code>
POST	<code>/insights/evaluate</code>
POST	<code>/insights/rule-data</code>
GET	<code>/health</code>
GET	<code>/health/stack</code> (admin only — enforced in controller)
GET	<code>/sources/presets</code>
GET	<code>/sources</code> , <code>/sources/{source}</code> , <code>/sources/{source}/data</code> , <code>/sources/{source}/summary</code> , <code>/sources/{source}/aggregate</code> ,

METHOD	PATH
	/sources/{source}/trends , /sources/{source}/runs
GET	/sites

Manager-only (admin or analyst)

METHOD	PATH
POST	/dashboards , /sources
PUT	/dashboards/{dashboard} , /sources/{source} , /sites/{site}
DELETE	/dashboards/{dashboard} , /sources/{source} , /sites/{site}
POST	/sources/{source}/refresh , /sources/{source}/unlock
POST	/sources/test
POST	/sites , /sites/{site}/assign

Admin-only (/admin/* , gated by EnsureRole middleware)

METHOD	PATH
GET	/admin/users , /admin/users/{user}
POST	/admin/users (create)
PUT	/admin/users/{user}
DELETE	/admin/users/{user}
POST	/admin/users/{user}/reset-password
POST	/admin/users/{user}/clear-ip-flag
POST	/admin/users/{user}/unlock
POST	/admin/users/{user}/disable-mfa
PUT	/admin/users/{user}/mfa-required
GET	/admin/dashboards (list assignable)
GET	/admin/users/{user}/dashboards
POST	/admin/users/{user}/dashboards
DELETE	/admin/users/{user}/dashboards/{dashboard}
GET	/admin/users/{user}/notification-subscriptions
PUT	/admin/users/{user}/notification-subscriptions
GET	/admin/mail-settings

METHOD	PATH
PUT	/admin/mail-settings
POST	/admin/mail-settings/test
GET	/admin/notification-templates
PUT	/admin/notification-templates/{template}
POST	/admin/notification-templates/{template}/reset
POST	/admin/notification-templates/{template}/preview
POST	/admin/notification-templates/{template}/test
GET	/admin/notification-logs
GET	/admin/login-events
GET	/admin/audit-logs

21. Frontend architecture

Routing

Next.js App Router. The `(app)` route group wraps every page in `<Shell>` (sidebar nav + topbar). Pages outside the group:

- `/login` — login + MFA challenge
- `/` — redirects authenticated users to `/dashboard`, anonymous to `/login`

State management

- **Auth:** `<AuthProvider>` context — single `useQuery(["me"])`. All

pages call `useAuth()` to access the user.

- **Server data:** TanStack Query everywhere. Query keys are stable

arrays so cache hits work across navigation.

- **Local UI:** plain `useState` / `useEffect`.

Shared UI primitives (`src/components/ui/`)

`Button`, `Card`, `Dialog`, `Input`, `Label`, `Select`, `Badge`, `Skeleton`, `Table`, `Tabs`. All variant-based via `class-variance-authority`.

Toast system

`src/lib/toast.tsx` exposes `useToast()`. Used across the app for mutation outcomes.

Variants: `success`, `error`, `warning`, `info`.

Type discipline

Every API response has a matching TypeScript interface in `lib/types.ts`. The

`apiFetch<T>` wrapper returns `Promise<T>` so callers get full inference. `tsc --noEmit` is clean.

22. Environment configuration

Backend `.env`

Critical keys (others use Laravel defaults):

```
APP_NAME="EDR Compliance Dashboard"
APP_ENV=production
APP_KEY=                                # php artisan key:generate
APP_DEBUG=false                        # MUST be false in prod
APP_URL=https://api.your-domain.com

FRONTEND_URL=https://app.your-domain.com
SANCTUM_STATEFUL_DOMAINS=app.your-domain.com
SESSION_DOMAIN=.your-domain.com # leading dot when sharing across subdomains

DATA_ENCRYPTION_KEY=                  # base64-encoded 32 bytes, see below
HASH_DRIVER=argon2id
ARGON_MEMORY=65536
ARGON_THREADS=1
ARGON_TIME=4

LOG_CHANNEL=stack
LOG_LEVEL=info

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=compliance_dashboard
DB_USERNAME=...
DB_PASSWORD=...

SESSION_DRIVER=database
SESSION_LIFETIME=120
SESSION_ENCRYPT=true
SESSION_PATH=/

CACHE_STORE=database
QUEUE_CONNECTION=database

# Mail can be left at log - the runtime DB override controls actual sending.
MAIL_MAILER=log
```

Generate `DATA_ENCRYPTION_KEY` :

```
php -r "echo 'base64:'.base64_encode(random_bytes(32)).PHP_EOL;"
```

Keep this key safe. Losing it means every SMTP password and every saved connector secret becomes undecryptable.

Frontend `.env.local`

```
NEXT_PUBLIC_API_URL=https://api.your-domain.com
```

`config/cors.php`

For SPA + cookies to work, you need:

```
'paths' => ['api/*', 'sanctum/csrf-cookie'],  
'allowed_origins' => ['https://app.your-domain.com'],  
'supports_credentials' => true,
```

23. Deployment

Server prerequisites (Linux VPS)

```
sudo apt update
sudo apt install -y nginx php8.3-fpm php8.3-
{mysql,mbstring,xml,curl,zip,bcmath,gd,intl} \
    mysql-server composer nodejs npm git unzip
# Verify: php -v (>=8.3), node -v (>=20), composer -V (>=2.4)
```

Best practice: deploy as the site user

On managed-panel hosting (CloudPanel, Plesk, cPanel) every site has its own Linux user that PHP-FPM and the Node process run as. Always run git, composer, npm, and artisan as that user — files created by root become unwritable by the web server.

```
sudo -i -u faisalkhan-apidashboard      # site user
cd ~/htdocs/api.your-domain.com         # site doc root (or one level deeper)
git clone https://github.com/samfaiz/ComplianceDashboardBackend.git .
cp .env.example .env
nano .env                                # fill in the values above
composer install --no-dev --optimize-autoloader
php artisan key:generate
php artisan migrate --force
php artisan db:seed --force              # seeds default users + notification
templates
php artisan config:cache && php artisan route:cache
exit
```



```
sudo -i -u faisalkhan-complainedashboard
cd ~/htdocs/app.your-domain.com
git clone https://github.com/samfaiz/ComplianceDashboardFrontend.git .
echo "NEXT_PUBLIC_API_URL=https://api.your-domain.com" > .env.local
npm ci
npm run build
# Run with PM2:
npm install -g pm2      # may need sudo depending on panel
pm2 start npm --name compliance-frontend -- start
pm2 save && pm2 startup
exit
```

Permissions (only if you fall back to root deploy)

```
chown -R faisalkhan-apidashboard:faisalkhan-apidashboard \
/home/faisalkhan-apidashboard/htdocs/api.your-domain.com
chmod -R 775 ../storage ../bootstrap/cache
```

Nginx

Two server blocks (one per subdomain). The backend block points `root` at `Backend/public/`. The frontend block reverse-proxies to `localhost:3000` (Next.js dev/start) or serves static `out/` if you `next export`. Get certificates with `sudo certbot --nginx`.

24. Update procedure

Once deployed, every code update is a short cycle. Always run as the site user, never as root.

```
# Database backup (cheap insurance)
mysqldump -u root -p compliance_dashboard > ~/backup-$(date +%F).sql

# Backend
sudo -i -u faisalkhan-apidashboard
cd ~/htdocs/api.your-domain.com
git pull origin main
composer install --no-dev --optimize-autoloader
php artisan migrate --force
php artisan db:seed --class=NotificationTemplateSeeder --force # idempotent
php artisan config:clear && php artisan route:clear
php artisan config:cache && php artisan route:cache
exit
sudo systemctl reload php8.3-fpm

# Frontend
sudo -i -u faisalkhan-complainedashboard
cd ~/htdocs/app.your-domain.com
git pull origin main
npm ci
npm run build
exit
pm2 restart compliance-frontend
```

After each update, verify:

```
curl -s -o /dev/null -w "%{http_code}\n" https://api.your-domain.com/api/login
# expect 405
curl -s -o /dev/null -w "%{http_code}\n" https://app.your-domain.com
# expect 200
```

25. Operations & gotchas

Common issues and fixes

SYMPTOM	CAUSE	FIX
<code>fatal: detected dubious ownership on git pull</code>	Repo is owned by a different user than the one running git	Run git as the file owner: <code>sudo -i -u <site-user></code> ; or <code>chown -R <site-user>:<site-user> .</code>
419 / CSRF errors on login	<code>SANCTUM_STATEFUL_DOMAINS</code> doesn't match the frontend host	Set to the exact hostname (no scheme, no port for 443/80)
Frontend can't reach API	Wrong <code>NEXT_PUBLIC_API_URL</code> , or CORS misconfigured	Fix env, restart Next.js; check <code>config/cors.php</code> allows the origin + credentials
<code>composer audit</code> failed in Tech Stack panel	<code>composer</code> not on PHP-FPM's PATH	Add to PHP-FPM pool's <code>env[PATH]</code> or symlink to <code>/usr/local/bin/</code>
<code>npm audit</code> failed likewise	<code>npm</code> not on PHP-FPM's PATH, or <code>Frontend/package-lock.json</code> not where the auditor expects	Same; or override <code>frontendPath()</code>
Login from new device not flagged	First IP for a user is auto-trusted (baseline)	This is by design — subsequent new IPs trigger flag + notification
MFA QR doesn't scan	System clock drift on the authenticator app	The TOTP window allows ± 1 step; if it still fails, fix the device clock
Email tests return success but nothing arrives	Transport is <code>log</code> ; "sent" means written to <code>storage/logs/laravel.log</code>	Switch transport to SMTP and enter credentials
Mail config UI changes don't take effect	<code>MailConfigurator</code> runs at boot; if FPM workers are warm they're using the old config	<code>sudo systemctl reload php8.3-fpm</code>

Recommended cron entries (optional)

```
# Run Laravel scheduler every minute (for any future Schedule::call entries)
* * * * * cd /home/<site>/htdocs/api.your-domain.com && php artisan schedule:run
>> /dev/null 2>&1

# Run a queued worker if you switch notifications to ShouldQueue
@reboot pm2 start "php artisan queue:work --tries=3" --name compliance-queue
```

Backups

- **Database** — daily `mysqldump` . Include the cache table if you care

about the tech-stack "seen advisories" set (it's regenerated on first scan after a wipe, but you'd get one spam burst).

- `.env` — back this up out-of-band. The `DATA_ENCRYPTION_KEY` is

load-bearing for SMTP passwords and saved connector secrets.

Logs

- `Backend/storage/logs/laravel.log` — application errors, mail in log

driver, notification failures.

- PHP-FPM error log (path varies by distro).
- Nginx access + error logs.

Scaling considerations

- The system is happily single-instance for small estates. Splitting the

queue out (`QUEUE_CONNECTION=redis` + a queue worker process) is the first thing to do once notifications get heavy.

- Snapshot retention is bounded — endpoints older than the retention

window are pruned (rolled-up summaries are kept indefinitely for trends).

- MySQL indexes already cover the hot queries (snapshot_id +

external_id on endpoints; user_id + is_default on dashboards; status + created_at on notification_logs).

26. Extending the system

Add a new notification event

1. Add an entry to `NotificationCatalog::events()` with `event_key`, default subject + body + variables + default audience.

1. Run `php artisan db:seed --class=NotificationTemplateSeeder` to seed the row (idempotent — won't overwrite admin edits).

1. From wherever the event happens in your code, call

```
app(NotificationService::class)->dispatch('your.event_key', $payload, $targetUser=null) .
```

That's it — the admin UI automatically lists the new event under its category, the Settings subscription card shows it, and the dispatch flow takes care of the rest.

Add a new widget type

1. Add the type to the `Widget["type"]` union in `Frontend/src/lib/types.ts`.

2. Add a branch in `WidgetBody` (`Frontend/src/components/widget.tsx`)

that renders it.

1. Add per-type knobs to the `WidgetConfig` interface and the

`WidgetConfigDialog` in `Frontend/src/app/(app)/builder/page.tsx`.

1. Add a `DEFAULT_SIZE` entry in the builder so new instances get sensible grid dimensions.

1. (Optional) Add a preset recipe to the `PRESETS` array.

Add a new connector vendor

1. Create `Backend/app/Services/Connectors/<Vendor>Connector.php` implementing the connector interface.

1. Register the vendor key in `ConnectorFactory`.

2. Add a preset in `ApiSourceController::presets()` so the UI wizard offers it.

1. Add the vendor key to the `Vendor` union in `Frontend/src/lib/types.ts`.

Add a new role

The system assumes three roles. Adding a fourth requires:

- Adding the constant on `User` + the `ROLES` array.
 - Updating `canManage()` / `isAdmin()` if relevant.
 - Updating the seed users and the admin "New user" form's role dropdown.
 - Adjusting the `RolesPermissionsCard` matrix.
 - Reviewing every `abort_unless($user->canManage(), 403)` site.
-

Appendix A — Default users (seeded)

```
admin@compliance.local / Admin@12345! → admin  
analyst@compliance.local / Analyst@12345! → analyst  
viewer@compliance.local / Viewer@12345! → viewer
```

All have `must_change_password=false` for convenience. Change them immediately on a production deployment.

Appendix B — Useful local commands

```
# Fresh DB + seed everything
php artisan migrate:fresh --seed --force

# Re-seed just the notification templates (idempotent)
php artisan db:seed --class=NotificationTemplateSeeder --force

# Inspect runtime mail config
php artisan tinker --execute='echo config("mail.default").PHP_EOL;'

# Force a fresh CVE scan
php artisan tinker --
execute='app(App\Services\Security\TechStackAuditor::class)->snapshot(true);'

# List all registered routes
php artisan route:list --path=api

# Clear caches between deploys
php artisan optimize:clear
```

End of document.